Ex.No.1 **Student Record Management System (Procedural Approach)**

Aim:

To design and implement a Student Record Management System in Python using a procedural programming approach to perform operations like adding, displaying, searching, deleting, and updating student records.

Description:

This Student Record Management System manages student information such as name, roll number, and marks in two subjects. It allows the user to add new student records, display all records, search for a student by roll number, delete a record, and update the roll number of a student. The system uses lists to store student data and functions to perform the required operations in a procedural manner.

Algorithm

Initialize an empty list to store student records.
Define functions for each operation:
Accept: Take input from the user for student details and add to the list.
Display: Show all student records in a formatted manner.
Search: Find a student by roll number and return the index.
Delete: Remove the student record by roll number.
Update: Modify the roll number of a student.
Use a menu-driven loop to allow the user to perform multiple operations until exiting.
Perform operations based on user choice.

Sample Coding (Procedural Approach in Python):

```
# Student Record Management System (Procedural Approach)

# List to hold student records
students = []

# Function to accept student details
def accept(name, rollno, marks1, marks2):
    student = {"name": name, "rollno": rollno, "marks1": marks1, "marks2": marks2}
    students.append(student)

# Function to display student details
def display():
    if not students:
        print("No records to display.")
        return
    for student in students:
        print(f"Name: {student['name']}")
        print(f"Roll No: {student['rollno']}")
        print(f"Marks1: {student['marks1']}")
        print(f"Marks2: {student['marks2']}")
        print()
```

```
# Function to search student by roll number
def search(rollno):
    for i, student in enumerate(students):
        if student["rollno"] == rollno:
            return i
    return -1

# Function to delete student by roll number
def delete(rollno):
    index = search(rollno)
    if index != -1:
        del students[index]
        print(f"Student with roll no {rollno} deleted.")
    else:
        print("Student not found.")

# Function to update student roll number
def update(old_rollno, new_rollno):
    index = search(old_rollno)
    if index != -1:
        students[index]["rollno"] = new_rollno
        print(f"Roll no updated from {old_rollno} to {new_rollno}.")
    else:
        print("Student not found.")

# Sample usage
accept("A", 1, 100, 100)
accept("B", 2, 90, 90)
accept("C", 3, 80, 80)

print("\nList of Students\n")
display()

print("Student Found:\n")
idx = search(2)
if idx != -1:
    student = students[idx]
    print(f"Name: {student['name']}")
    print(f"Roll No: {student['rollno']}")
    print(f"Marks1: {student['marks1']}")
    print(f"Marks2: {student['marks2']}\n")

delete(2)

print(f"\nList after deletion ({len(students)} students):\n")
display()

update(3, 2)

print(f"\nList after updation:\n")
display()
```

Sample Output:

List of Students

Name: A
Roll No: 1
Marks1: 100
Marks2: 100

Name: B
Roll No: 2
Marks1: 90
Marks2: 90

Name: C
Roll No: 3
Marks1: 80
Marks2: 80

Student Found:

Name: B
Roll No: 2
Marks1: 90
Marks2: 90

Student with roll no 2 deleted.

List after deletion (2 students):

Name: A
Roll No: 1
Marks1: 100
Marks2: 100

Name: C
Roll No: 3
Marks1: 80
Marks2: 80

Roll no updated from 3 to 2.

List after updation:

Name: A
Roll No: 1
Marks1: 100
Marks2: 100

Name: C
Roll No: 2
Marks1: 80
Marks2: 80

Ex.No.2 **Bank Account Simulation Using OOP**

Aim

To develop a simple bank account simulation using Object-Oriented Programming (OOP) concepts in Python that supports basic banking operations like deposit, withdrawal, and displaying balance.

Description

This lab exercise demonstrates the implementation of a BankAccount class in Python using OOP principles. The class will encapsulate attributes such as account holder name and balance, and methods for depositing money, withdrawing money, and displaying the current balance. This simulation helps understand core OOP concepts such as encapsulation, methods, and class instantiation in a practical use case.

Algorithm:

1. Start
2. Initialization:
3. Create an instance of BankAccount with account holder name and initial balance (default 0.0).
4. Deposit method:
5. Accept deposit amount.
6. Check if the amount is positive.
7. If yes, add the amount to the balance.
8. Display the deposited amount and new balance.
9. If no, display an error message for invalid deposit amount.
10. Withdraw method:
11. Accept withdrawal amount.
12. Check if the amount is positive.
13. If no, display an error message.
14. Else, check if the balance is sufficient to cover the withdrawal.
15. If yes, deduct the amount from the balance.
16. Display the withdrawn amount and new balance.
17. If no, display "Insufficient funds" message.
18. Display balance method:
19. Print the account holder's name and the current balance.
20. End

Code

```
class BankAccount:
    """Class to simulate a bank account with deposit and withdrawal functionality."""

    def __init__(self, account_holder, balance=0.0):
        """Initialize account holder name and starting balance."""
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        """Deposit money into the account if amount is positive."""
        if amount > 0:
            self.balance += amount
            print(f"Deposited ${amount:.2f}. New balance: ${self.balance:.2f}")
        else:
            print("Deposit amount must be positive.")
```

```
    def withdraw(self, amount):
        """Withdraw money from the account if enough balance is available."""
        if amount <= 0:
            print("Withdrawal amount must be positive.")
        elif amount > self.balance:
            print("Insufficient funds.")
        else:
            self.balance -= amount
            print(f"Withdrew ${amount:.2f}. New balance: ${self.balance:.2f}")

    def display_balance(self):
        """Display the current balance and account holder information."""
        print(f"Account Holder: {self.account_holder}, Balance: ${self.balance:.2f}")

# Creating an account and performing transactions
account = BankAccount("John Doe", 1000.0)

account.display_balance()
account.deposit(500)
account.withdraw(300)
account.withdraw(1500)  # This should show "Insufficient funds"
account.display_balance()
```

Expected Output

```
Account Holder: John Doe, Balance: $1000.00
Deposited $500.00. New balance: $1500.00
Withdrew $300.00. New balance: $1200.00
Insufficient funds.
Account Holder: John Doe, Balance: $1200.00
```

Ex.No.3 **Implement Custom Stack and Queue ADTs with Recursion and Algorithm Analysis**

Aim

To understand and implement recursive algorithms in Python and analyze their time complexity.

Description

Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem. Recursive solutions consist of a base case (termination condition) and a recursive case (the function calls itself with smaller inputs). Algorithm analysis evaluates the efficiency of these recursive functions, often through time complexity. This lab covers examples such as factorial computation, Fibonacci series, and basic recursive algorithms, demonstrating both their implementation and performance considerations.

Algorithm:

1. Start
2. Define function factorial(n)
3. Check if n equals 1 (base case)
4. If yes, return 1
5. Otherwise (recursive case), return n multiplied by factorial(n - 1)
6. End function
7. Call factorial function with desired number (e.g., 5)
8. Print the result (factorial of 5 is 120)

Code Example (Factorial using Recursion)

```
def factorial(n):
    if n == 1:
        return 1  # base case
    else:
        return n * factorial(n-1)  # recursive case

# Test
num = 5
print(f"Factorial of {num} is {factorial(num)}")
```

Output

Factorial of 5 is 120

Ex.No.4 **Implement Custom Stack and Queue ADTs with Recursion and Algorithm Analysis**

Aim

To implement a recursive merge sort algorithm in Python that counts the number of operations performed and supports sorting of custom objects.

Description

Merge Sort is a classic divide-and-conquer sorting algorithm that recursively divides the input list into halves until single elements remain, then merges those halves back together in sorted order. Recursive calls allow breaking down the problem, and the merging process combines sorted lists. Operation counting can measure the number of key comparisons or steps taken, useful for analysis. The algorithm can be adapted to sort lists of custom objects by passing a comparison function or using object attributes for ordering.

Algorithm:

1. Start
2. Define a function merge_sort(arr, compare_func=None) to sort the list arr.
3. Initialize an operation counter ops to keep track of comparisons.
4. Define an inner function merge(left, right):
5. Create an empty list merged.
6. Use two pointers i and j to traverse left and right.
7. While both pointers are within bounds:
8. Increment operation count.
9. Compare elements using compare_func if provided, otherwise use default comparison.
10. Append the smaller element to merged and increment corresponding pointer.
11. Append any remaining elements from left and right to merged.
12. Return merged.
13. Define an inner recursive function recursive_merge_sort(lst):
14. If list length ≤ 1, return lst (base case).
15. Find midpoint and split list into left and right.
16. Recursively sort left and right.
17. Merge sorted halves using merge function.
18. Call recursive_merge_sort(arr) to get sorted list.
19. Return the sorted list and the total operation count.
20. End

Code with Operation Count and Custom Object Sorting

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __repr__(self):
        return f"Product({self.name}, {self.price})"

def merge_sort(arr, compare_func=None):
    """
    Recursive merge sort that counts operations and supports custom comparison.
```

```
    :param arr: List of items to sort
    :param compare_func: Function to compare two elements, returns True if first < second
    :return: (sorted list, operation count)
    """
    # Global operation count holder via closure
    ops = {'count': 0}

    def merge(left, right):
        merged = []
        i = j = 0
        while i < len(left) and j < len(right):
            ops['count'] += 1
            if compare_func:
                if compare_func(left[i], right[j]):
                    merged.append(left[i])
                    i += 1
                else:
                    merged.append(right[j])
                    j += 1
            else:
                if left[i] < right[j]:
                    merged.append(left[i])
                    i += 1
                else:
                    merged.append(right[j])
                    j += 1
        # Collect remaining elements
        merged.extend(left[i:])
        merged.extend(right[j:])
        return merged

    def recursive_merge_sort(lst):
        if len(lst) <= 1:
            return lst
        mid = len(lst) // 2
        left_sorted = recursive_merge_sort(lst[:mid])
        right_sorted = recursive_merge_sort(lst[mid:])
        return merge(left_sorted, right_sorted)

    sorted_arr = recursive_merge_sort(arr)
    return sorted_arr, ops['count']

# Example usage 1: Sorting integers
arr = [12, 11, 13, 5, 6, 7]
sorted_arr, op_count = merge_sort(arr)
print("Sorted array:", sorted_arr)
print("Operation count:", op_count)

# Example usage 2: Sorting custom objects by price
products = [
    Product("Laptop", 1200),
    Product("Phone", 800),
    Product("Tablet", 400),
    Product("Monitor", 500)
```

```
]

# Custom comparison function for products by price
def compare_products(p1, p2):
    return p1.price < p2.price

sorted_products, prod_op_count = merge_sort(products, compare_products)
print("Sorted products by price:", sorted_products)
print("Operation count:", prod_op_count)
```

Output:

```
Sorted array: [5, 6, 7, 11, 12, 13]
Operation count: 12
Sorted products by price: [Product(Tablet, 400), Product(Monitor, 500), Product(Phone, 800),
Product(Laptop, 1200)]
Operation count: 5
```

Ex.No.5 Implement Breadth-First Search (BFS), Dept-First Search (DFS)

Aim:
To implement the Breadth-First Search (BFS) algorithm in Python to traverse a graph systematically, visiting all the vertices level by level.

Description
Breadth-First Search (BFS) is a graph traversal algorithm that starts from a selected node and explores all its neighbors at the current depth prior to moving on to nodes at the next depth level. BFS uses a queue data structure for keeping track of nodes to visit next and marks nodes as visited to avoid revisiting. It is widely used in shortest path algorithms, peer-to-peer networks, and web crawling.

Algorithm
1. Initialize a queue and add the starting node to it.
2. Mark the starting node as visited.
3. While the queue is not empty:
4. Remove the front node from the queue.
5. Visit the node.
6. Add all unvisited neighbors of the node to the queue and mark them as visited.

Python Code
```
def bfs(graph, start_node):
    visited = []
    queue = []

    visited.append(start_node)
    queue.append(start_node)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.append(neighbor)
                queue.append(neighbor)

# Example graph represented as adjacency list
graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

print("BFS traversal starting from node 5:")
bfs(graph, '5')
```

Output:
```
BFS traversal starting from node 5:
5 3 7 2 4 8
```

Depth-First Search (DFS) Lab Manual

Aim

To implement the Depth-First Search (DFS) algorithm in Python to traverse a graph or tree by exploring **as far along a branch as possible before backtracking.**

Description

Depth-First Search (DFS) is a graph traversal method that explores a branch fully before backtracking and visiting other branches. DFS uses recursion or an explicit stack to keep track of nodes. It is useful for connectivity, pathfinding, and topological sorting.

Algorithm

1. Start from the selected node and mark it as visited.
2. Recursively visit each unvisited neighbor of the current node.
3. Backtrack when no unvisited neighbors remain.

Python Code (Recursive)

```
   def dfs(graph, node, visited=None):
   if visited is None:
      visited = set()
   visited.add(node)
   print(node, end=" ")
   for neighbor in graph[node]:
      if neighbor not in visited:
         dfs(graph, neighbor, visited)

# Example graph represented as adjacency list
graph = {
   '5': ['3', '7'],
   '3': ['2', '4'],
   '7': ['8'],
   '2': [],
   '4': ['8'],
   '8': []
}

print("\nDFS traversal starting from node 5:")
dfs(graph, '5')
```

Output

DFS traversal starting from node 5:
5 3 2 4 8 7

Ex.No.6 Implement Dijkstra's Algorithm for shortest path (Greedy)

Aim:

To implement Dijkstra's Algorithm to find the shortest path from a source vertex to all other vertices in a weighted graph using the greedy approach.

Description:

Dijkstra's Algorithm is a popular greedy algorithm used to find the shortest path between nodes

in a graph with non-negative edge weights. It works by building a shortest path tree from the source vertex, repeatedly selecting the vertex with the minimum tentative distance, and updating the distances to its adjacent vertices. This ensures the shortest paths are found efficiently.

Algorithm:

1. Initialize distances of all vertices from the source as infinity, except the source itself which is zero.
2. Set of vertices included in the shortest path tree (SPT) is initially empty.
3. Repeat until all vertices are included in SPT:
4. Pick the vertex u not in SPT with the smallest distance.
5. Add u to the SPT.
6. Update the distance values of all adjacent vertices of u. For each adjacent vertex v, if the distance to u plus the edge weight u-v is less than the current distance to v, update it.
7. When all vertices are processed, the distance array contains the shortest distances from the source.

Python Code Implementation:

```
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.graph = [[0 for _ in range(vertices)] for _ in range(vertices)]

    def printSolution(self, dist):
        print("Vertex \t Distance from Source")
        for node in range(self.V):
            print(node, "\t\t", dist[node])

    def minDistance(self, dist, sptSet):
        min_val = float('inf')
        min_index = -1

        for v in range(self.V):
            if dist[v] < min_val and not sptSet[v]:
                min_val = dist[v]
                min_index = v

        return min_index

    def dijkstra(self, src):
        dist = [float('inf')] * self.V
        dist[src] = 0
        sptSet = [False] * self.V

        for _ in range(self.V):
            u = self.minDistance(dist, sptSet)
            sptSet[u] = True

            for v in range(self.V):
                if self.graph[u][v] > 0 and not sptSet[v] and dist[v] > dist[u] + self.graph[u][v]:
                    dist[v] = dist[u] + self.graph[u][v]

        self.printSolution(dist)

# Example usage:
g = Graph(9)
g.graph = [
```

```
    [0, 4, 0, 0, 0, 0, 0, 8, 0],
    [4, 0, 8, 0, 0, 0, 0, 11, 0],
    [0, 8, 0, 7, 0, 4, 0, 0, 2],
    [0, 0, 7, 0, 9, 14, 0, 0, 0],
    [0, 0, 0, 9, 0, 10, 0, 0, 0],
    [0, 0, 4, 14, 10, 0, 2, 0, 0],
    [0, 0, 0, 0, 0, 2, 0, 1, 6],
    [8, 11, 0, 0, 0, 0, 1, 0, 7],
    [0, 0, 2, 0, 0, 0, 6, 7, 0]
]

g.dijkstra(0)
```

Output:

Vertex	Distance from Source
0	**0**
1	**4**
2	**12**
3	**19**
4	**21**
5	**11**
6	**9**
7	**8**
8	**14**

Ex.No.7 Implement sorting algorithms in Python Bubble Sort, Selection Sort

Bubble Sort

Aim
To implement the Bubble Sort algorithm in Python to sort a list of numbers in ascending order.

Description
Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until no more swaps are needed, which means the list is sorted.

Algorithm
1. Start from the first element of the list.
2. Compare the current element with the next element.
3. If the current element is greater than the next element, swap them.
4. Move to the next element and repeat steps 2-3 until the end of the list.
5. Repeat the above process for all elements until no swaps are needed.

Python Code

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        swapped = False
        for j in range(n-1-i):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90, 5]
print("Before sorting:", arr)
bubble_sort(arr)
print("After sorting:", arr)
```

Output
Before sorting: [64, 34, 25, 12, 22, 11, 90, 5]
After sorting: [5, 11, 12, 22, 25, 34, 64, 90]

Selection Sort
Aim
To implement the Selection Sort algorithm in Python to sort a list of numbers in ascending order.
Description

Selection Sort algorithm sorts an array by repeatedly finding the minimum element from the unsorted part and swapping it with the first unsorted element. This way, the sorted portion grows from the front of the array.

Algorithm

1. Set the first element as the minimum.
2. Compare this minimum with the rest of the array.
3. If a smaller element is found, update the minimum.
4. After one pass, swap the minimum element with the first unsorted element.
5. Move to the next element and repeat steps 1-4 until the array is sorted.

Python Code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

# Example usage
arr = [64, 25, 12, 22, 11]
print("Before sorting:", arr)
selection_sort(arr)
print("After sorting:", arr)
```

Output

```
Before sorting: [64, 25, 12, 22, 11]
After sorting: [11, 12, 22, 25, 64]
```

Ex.No.8 **Implement Linear Search, Binary Search, Hash Table with Collision Handling**

Linear Search

Aim:
To implement Linear Search algorithm in Python to find the position of an element in a given list.

Description:
Linear Search is a simple searching algorithm that checks every element in the list sequentially to find a target element. It is useful for unsorted lists.

Algorithm:

1. Start from the first element of the list.
2. Compare the current element with the target element.
3. If it matches, return the current index.
4. Else, move to the next element.
5. Repeat steps 2-4 until the end of the list.
6. If the element is not found, return -1.

Code:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Sample Input
arr = [5, 8, 1, 3, 7, 9]
target = 3

# Function Call and Output
index = linear_search(arr, target)
if index != -1:
    print(f"Element {target} found at index {index}")
else:
    print(f"Element {target} not found")
```

Output:
Eleme Element 3 found at index 3nt 3 found at index 3

Binary Search

Aim:
To implement Binary Search algorithm in Python to find the position of an element in a sorted list.

Description:
Binary Search works on a sorted list by repeatedly dividing the search interval in half. It compares the middle element of the interval with the target value to decide whether to continue the search in the left or right half.

Algorithm:

1. Initialize two pointers: left = 0 and right = n-1 (where n is list size).
2. While left is less than or equal to right:
3. Calculate mid = (left + right) // 2.
4. If element at mid equals the target, return mid.
5. If element at mid is less than target, set left = mid + 1.
6. Else set right = mid - 1.
7. If element not found, return -1.

Code:

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Sample Input
arr = [1, 3, 5, 7, 9, 11, 13]
target = 7

# Function Call and Output
index = binary_search(arr, target)
if index != -1:
    print(f"Element {target} found at index {index}")
else:
    print(f"Element {target} not found")
```

Code:

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
```

```
            right = mid - 1
    return -1

# Sample Input
arr = [1, 3, 5, 7, 9, 11, 13]
target = 7

# Function Call and Output
index = binary_search(arr, target)
if index != -1:
    print(f"Element {target} found at index {index}")
else:
    print(f"Element {target} not found")
```

Output:

Element 7 found at index 3

Hash Table with Collision Handling (Using Linear Probing)

Aim:

To implement a Hash Table with collision handling using linear probing in Python.

Description:

A Hash Table stores data in key-value pairs where a hash function determines the index for storing the value. Collisions occur when multiple keys hash to the same index. Linear probing handles collisions by checking the next available slot sequentially.

Algorithm:

1. Insert: Calculate hash index. If index is occupied, move linearly until an empty slot is found. Insert there.
2. Search: Calculate hash index. If key at index matches, return value. Else move linearly to next slots until found or empty slot reached.
3. Delete: Search for key and remove the key-value pair by setting slot to None.

Code:

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def hash_function(self, key):
        return key % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        start_index = index

        while self.table[index] is not None and self.table[index][0] != key:
            index = (index + 1) % self.size
            if index == start_index:  # Table full
                print("Hash table is full")
                return
        self.table[index] = (key, value)

    def search(self, key):
        index = self.hash_function(key)
```

```
        start_index = index

        while self.table[index] is not None:
            if self.table[index][0] == key:
                return self.table[index][1]
            index = (index + 1) % self.size
            if index == start_index:
                break
        return None

    def delete(self, key):
        index = self.hash_function(key)
        start_index = index

        while self.table[index] is not None:
            if self.table[index][0] == key:
                self.table[index] = None
                return True
            index = (index + 1) % self.size
            if index == start_index:
                break
        return False

    def display(self):
        for i, val in enumerate(self.table):
            if val is not None:
                print(f"Index {i}: Key = {val[0]}, Value = {val[1]}")

# Sample Usage
ht = HashTable(7)
ht.insert(10, 'Apple')
ht.insert(20, 'Banana')
ht.insert(15, 'Cherry')  # Causes collision with 15 % 7 = 1 (collision with 20)
print("Searching for key 20:", ht.search(20))
ht.delete(10)
ht.display()
```

Output:

```
Searching for key 20: Banana
Index 1: Key = 20, Value = Banana
Index 2: Key = 15, Value = Cherry
```

Ex.No.:9 **Implement Binary Tree ADT**

Aim

To implement a Binary Tree Abstract Data Type (ADT) in Python with basic operations and traversals.

Description

A Binary Tree is a hierarchical data structure in which each node has at most two children called the left child and the right child. Each node contains a data element and references to its left and right child nodes. Binary Trees are used in various applications such as expression parsing, hierarchical data representation, and binary search trees.

Algorithm

1. Define a Node class with data, left, and right pointers initialized to None.
2. Define a BinaryTree class with:
3. An initializer to set the root node to None.
4. Insert operation to add nodes to the tree (for example, insert in level order or based on some property).
5. Methods for traversals: inorder, preorder, and postorder.
6. For each traversal:
7. If the node is None, return.
8. Otherwise, recursively visit nodes in the defined order.
9. Display traversal results to verify the tree structure.

Python Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = Node(data)
        if not self.root:
            self.root = new_node
            return
        # Level order insertion using queue
        queue = [self.root]
        while queue:
            temp = queue.pop(0)
            if not temp.left:
                temp.left = new_node
                return
            else:
                queue.append(temp.left)
            if not temp.right:
                temp.right = new_node
                return
            else:
```

```
            queue.append(temp.right)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=' ')
            self.inorder(node.right)

    def preorder(self, node):
        if node:
            print(node.data, end=' ')
            self.preorder(node.left)
            self.preorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=' ')

# Sample usage and output
bt = BinaryTree()
for value in [1, 2, 3, 4, 5, 6, 7]:
    bt.insert(value)

print("Inorder traversal: ", end='')
bt.inorder(bt.root)
print()

print("Preorder traversal: ", end='')
bt.preorder(bt.root)
print()

print("Postorder traversal: ", end='')
bt.postorder(bt.root)
print()
```

Output:

```
Inorder traversal: 4 2 5 1 6 3 7
Preorder traversal: 1 2 4 5 3 6 7
Postorder traversal: 4 5 2 6 7 3 1
```

1. Start
2. Define a function `merge_sort(arr, compare_func=None)` to sort the list `arr`.

3. Initialize an operation counter `ops` to keep track of comparisons.
4. Define an inner function `merge(left, right)`:
- Create an empty list `merged`.
- Use two pointers `i` and `j` to traverse `left` and `right`.
- While both pointers are within bounds:
- Increment operation count.
- Compare elements using `compare_func` if provided, otherwise use default comparison.
- Append the smaller element to `merged` and increment corresponding pointer.
- Append any remaining elements from `left` and `right` to `merged`.
- Return `merged`.
5. Define an inner recursive function `recursive_merge_sort(lst)`:
- If list length ≤ 1, return `lst` (base case).
- Find midpoint and split list into `left` and `right`.
- Recursively sort `left` and `right`.
- Merge sorted halves using `merge` function.
6. Call `recursive_merge_sort(arr)` to get sorted list.
7. Return the sorted list and the total operation count.
8. End

Ex.No.:10 **Implement an AVLTree class with Rotations (LL, RR, LR, RL)**

Aim

To implement an AVL Tree class in Python that maintains a balanced binary search tree using rotations: Left-Left (LL), Right-Right (RR), Left-Right (LR), and Right-Left (RL) rotations.

Description

AVL Tree is a self-balancing binary search tree where the height difference (balance factor) between left and right subtrees for every node is at most 1. Imbalance after insertion or deletion is corrected using rotations (LL, RR, LR, RL) to maintain the balance, ensuring O(log n) time complexity for search, insert, and delete operations.

Algorithm

Insertion:

Insert the node as in a standard BST.
Update the height of all ancestor nodes.
Check the balance factor (height of left subtree - height of right subtree) for every node.
If the balance factor is outside the range [-1, 1], apply one of the rotations:
LL Rotation: Left subtree of left child causes imbalance.
RR Rotation: Right subtree of right child causes imbalance.
LR Rotation: Right subtree of left child causes imbalance (double rotation: left rotate left child, right rotate node).
RL Rotation: Left subtree of right child causes imbalance (double rotation: right rotate right child, left rotate node).

Rotations:

Left Rotation (RR):
Rotate the subtree right to the left.
Right Rotation (LL):
Rotate the subtree left to the right.

Left-Right Rotation (LR):
Left rotate left subtree, then right rotate root.
Right-Left Rotation (RL):
Right rotate right subtree, then left rotate root.

Code Implementation

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def getHeight(self, node):
        if not node:
            return 0
        return node.height

    def getBalance(self, node):
        if not node:
            return 0
        return self.getHeight(node.left) - self.getHeight(node.right)

    def rightRotate(self, y):
        x = y.left
        T2 = x.right

        # Perform rotation
        x.right = y
        y.left = T2

        # Update heights
        y.height = 1 + max(self.getHeight(y.left), self.getHeight(y.right))
        x.height = 1 + max(self.getHeight(x.left), self.getHeight(x.right))

        return x

    def leftRotate(self, x):
        y = x.right
        T2 = y.left

        # Perform rotation
        y.left = x
        x.right = T2

        # Update heights
        x.height = 1 + max(self.getHeight(x.left), self.getHeight(x.right))
        y.height = 1 + max(self.getHeight(y.left), self.getHeight(y.right))

        return y

    def insert(self, node, data):
```

```
        # Normal BST insertion
        if not node:
            return TreeNode(data)
        elif data < node.data:
            node.left = self.insert(node.left, data)
        else:
            node.right = self.insert(node.right, data)

        # Update the ancestor node height
        node.height = 1 + max(self.getHeight(node.left), self.getHeight(node.right))

        # Get the balance factor
        balance = self.getBalance(node)

        # If node is unbalanced, then perform rotations

        # Left Left Case
        if balance > 1 and data < node.left.data:
            return self.rightRotate(node)

        # Right Right Case
        if balance < -1 and data > node.right.data:
            return self.leftRotate(node)

        # Left Right Case
        if balance > 1 and data > node.left.data:
            node.left = self.leftRotate(node.left)
            return self.rightRotate(node)

        # Right Left Case
        if balance < -1 and data < node.right.data:
            node.right = self.rightRotate(node.right)
            return self.leftRotate(node)

        return node

    def inOrderTraversal(self, root):
        if root is not None:
            self.inOrderTraversal(root.left)
            print(root.data, end=" ")
            self.inOrderTraversal(root.right)

# Usage example:
avl = AVLTree()
root = None
values = [10, 20, 30, 40, 50, 25]

for v in values:
    root = avl.insert(root, v)

print("Inorder traversal of the constructed AVL tree is:")
avl.inOrderTraversal(root)
```

Output:
Inorder traversal of the constructed AVL tree is:
10 20 25 30 40 50